

PROJECT LUMOS

Draw your room, put it on the map, and see where the sunlight will actually fall in it, hour by hour, before you buy the furniture.

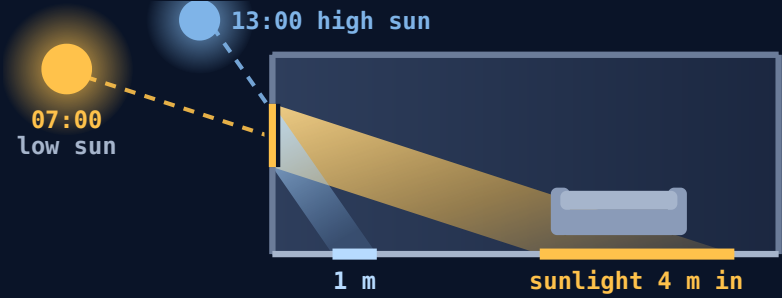
Sumin · Darlene · Matthew

Repository
github.com/tjandera/lumos
MIT licensed

Built with
TypeScript · React 19 · three.js
Vite · Fastify · Node 22 · Postgres

Infrastructure
pnpm monorepo, 4 packages + 2 apps
Docker · Kubernetes · Vitest

AI
OpenAI GPT API key drives
image generation and photo import



Code and
write-up

The one thing the app computes. A low sun throws daylight deep into a room; a high sun lights only the strip by the window. Which one you get depends on your latitude, your date, and which way the building faces.

THE CLAIM WE ARE HERE TO PROVE

A browser tool can answer both questions that decide a room, **will it fit** and **what will the light be like**, and we measured both answers instead of asserting them.

0.219°
SUN, VS THE NOAA
REFERENCE MODEL

89%→15%
BROKEN LAYOUT
RULES,
RANDOM VS OUR
PLACER

788
TESTS
PASSING

25
TESTS FAILING,
CI IS RED

1 PROBLEM You commit to a room you saw once, at one hour, in one season.

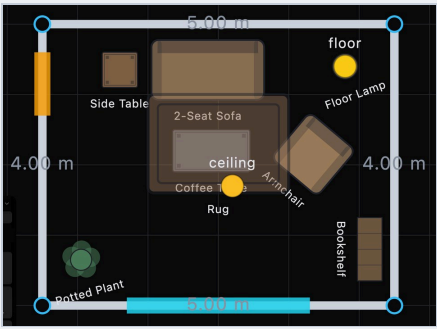
A listing photo answers none of the questions that decide the money:

- Will direct sun reach this room in the afternoon, or only grey light?
- Will this sofa sit in a sunbeam that fades it?
- Is the desk bright enough to work at, or is that a guess?
- Will the sofa physically fit and still leave room to walk past?

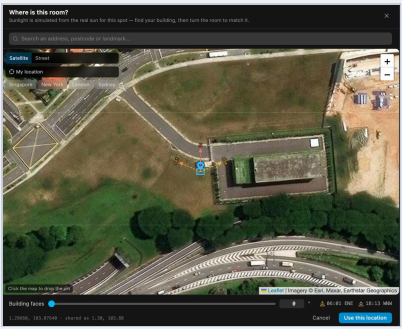
Who it is for

The **move-in shopper**: someone about to commit to a layout or a purchase for one specific room, who wants the answer before delivery day and will not install CAD software to get it.

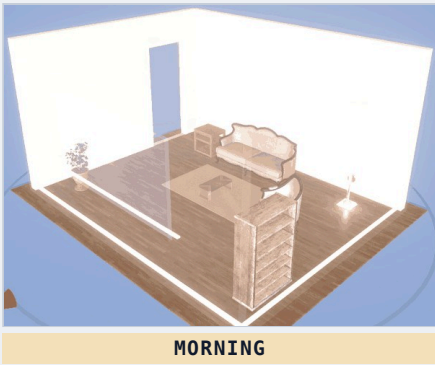
2 THE PRODUCT Purpose, and how it works.



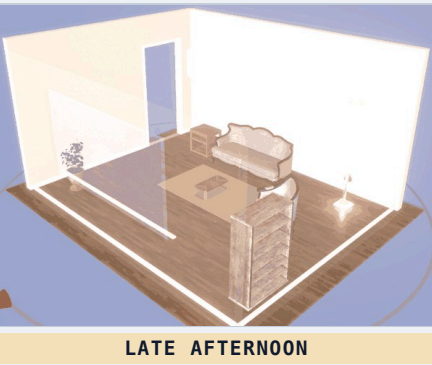
1. Draw the room.
Real walls, windows, doors.



2. Say where it is.
Drop a pin, turn it to true north.

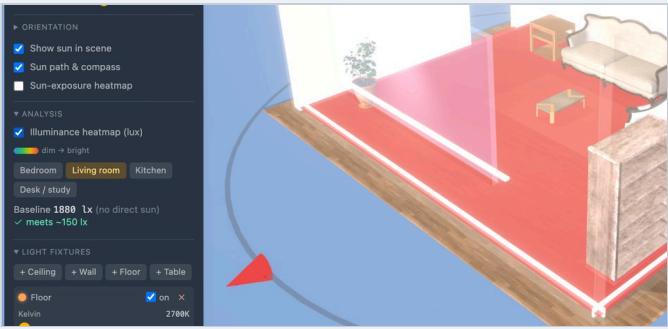


3. Scrub the day.
Same room and camera.
Only the time changed.



MORNING

LATE AFTERNOON



4. Read the floor.
Brightness in lux, checked
against a room standard.

The purpose: answer all four questions above in a browser, for that flat rather than a showroom, before any money changes hands. No CAD install, no account, and the core loop costs nothing to run.

Also in it: 35 catalogue pieces at verified real sizes, filtered by what fits the space you drew. Eight fixture types with real colour temperature and brightness. Blinds that block light. Export a file, or share a link that carries no address.

3 THE SCIENCE How the sun calculation works, and how accurate it is.

The science

Give it a latitude, a longitude, a date and a time and there is exactly **one correct answer** for where the sun is. We compute that from the standard astronomical formulas, then rotate it by how far the building is turned from true north.

sun = f(lat, lng, date, time), then rotated by north offset

The 3D engine then casts shadows from a light pointing exactly that way, so the shadows on your floor are **predictions you can check by standing in the room**. Bounced light comes from a photographed 360° panorama of a real apartment rather than a flat grey fill.

Brightness on the floor is real units, not a look:

lux = I · cosθ / d²

summed over daylight and every fixture, then compared against the level a room of that type needs.

What we considered

OPTION	VERDICT
A mood preset what consumer planners do	Rejected. Looks nice, says nothing about your flat.
Offline path tracing the best physics	Rejected. Cannot scrub a day at 60 fps in a browser.
Baked lighting pre-computed, fast	Rejected. Right for one layout, wrong once a chair moves.
Real-time shadows on a geographic sun	Chosen. Stays true while furniture moves.

What it gives you

- A time-of-day slider, and a 24-hour study at one render per hour
- Sun path and compass drawn for your actual site
- A floor heatmap in lux, checked against bedroom, living room, kitchen or desk levels, with blinds that dim the heatmap too

The metrics

We rebuilt NOAA's published sun-position equations independently, sharing no code with the app, and compared. Every 10 minutes, all of 2026, three latitudes, **70,475 daylight readings**.

SITE	READINGS	WORST ERROR
Singapore 1.35°N	24,734	0.219°
New York 40.71°N	24,301	0.219°
Reykjavik 64.15°N	21,440	0.219°

Why 0.219° is small. The sun is 0.533° wide in the sky, so our worst error is **41% of the sun's own width**. It moves a shadow **3.8 mm at one metre**, where the shadow's soft edge is already **9.3 mm** wide.

It is also checked against known answers: overhead at the equator at equinox noon (>85°), low in Oslo at midwinter (0–12°), below the horizon at midnight.

4 APPROACH The AI is never allowed to say where things go.

Ask a language model to arrange a room and it returns positions that look reasonable and put the sofa through the wall. So it never returns positions. It describes what it wants, and ordinary rule-based code does the placing.

OPTION	WHY REJECTED, OR CHOSEN
The model gives coordinates	Rejected. Nothing checks it, failures are silent, and the same request gives a different room each time.
Fixed rules only, no model	Rejected. Cannot understand "cozy", "a reading corner", or "under \$3,000".
Model describes, solver places	Chosen. The model reads intent; the code satisfies the hard rules.

The placer checks every candidate spot against every rule: inside the room, not overlapping, walkway kept clear, door able to swing. It returns a checked position, or **it says no and explains why**.

One file is the contract

Everything the app knows about a room lives in one plain JSON file. The 3D view draws it, the AI edits it, the server stores it, and nothing else is shared. Each version ships an upgrader, so **a design saved months ago still opens**. Your address is never in the file, only a coordinate rounded to about a kilometre.

6 CONSTRAINTS The limits that shaped the build.

- Speed.** The 3D view targets **60 frames per second** and holds a budget of 300 draw calls and 500,000 triangles. An on-screen meter (F9) shows it live. Under 40 fps for ten seconds, lighting quality drops one step automatically.
- Cost and latency.** Drawing, furnishing and scrubbing the day cost **nothing**: no key, no network. Image generation calls the GPT API at **35 to 90 seconds a call**, which ruled out most free hosting tiers, so it ships with a free offline mode.
- Privacy.** Your home address never enters the file. Only a coordinate rounded to **about 1 km** travels with a design, and the rounding happens **on the server**, so a modified browser cannot bypass it. Photo GPS data stays on your machine.
- Abuse.** The image endpoints have no login by design, so a public link without a daily cap is a free image generator on our bill. A daily ceiling and a spend limit contain it.
- Reliability and reach.** If the graphics context is lost the app recovers instead of going white, and saves are written atomically so an interrupted save cannot corrupt a design. Panels become bottom sheets on phones.

RUN IT, OR CHECK OUR SUN YOURSELF

```
pnpm install && pnpm dev # core loop needs no key
node scripts/validation/sun-direction-error.js
```

5 EVIDENCE Does the layout actually hold together?

200 seeded trials, 3 to 6 items each, rooms 3.5 to 7 m wide. The same 885 placement requests went to our placer and to a **random-placement baseline**, graded by one checker written separately from the placer. The workload includes an item too big to fit, so refusals reflect real failures.

PLACEMENTS THAT BROKE A RULE	RANDOM	OURS
Any rule broken	89.2%	14.9%
Outside the room	50.5%	0%
Overlapping another item	74.8%	0%
Walkway blocked	44.6%	14.9%
Door swing blocked	26.2%	0%
Said no, rather than placing badly	0%	19.1%

Three of four failure types go to zero. The fourth is a real defect we found this way and left in (section 7). **The baseline is random placement, not a language model**: it shows these rules are hard to satisfy by chance, not that we beat an LLM.

Suite: core 133 · renderer 55 · ai 50 · api 230 · web 320 = **788 passing, 25 failing** (section 7).

7 HONESTY Where it fails, and what we have not checked.

The defect our own benchmark found

14.9% of placements leave a walkway blocked, which the placer's own documentation says cannot happen. **Cause:** it remembers each item's footprint but not the clear space in front of it, so item 2 can land in the walkway item 1 reserved. All 107 violations have that shape. **The tests missed it:** they check each item against the room as it was at the time, the same blind spot the placer has. We left it in because the measurement is the result.

- Time zones are not verified.** Daylight-saving handling has never been checked. In a region that observes it, shadows could be an hour out, rendered confidently.
- The test suite is red.** 25 tests fail after a module move still in progress, so pnpm test exits non-zero. The tests point at old paths; the features work.
- Not measured.** No real language model benchmarked against our placer. Postgres tested against an imitation, Docker never run. **No user study: zero outside users.**

Given two more weeks

- Fix the walkway defect and re-run this benchmark. The harness exists, so it is verifiable the day it lands.
- Add tests that grade a finished room as a whole, not item by item.
- Check time zones, then watch five real move-in shoppers use it.



ALL SOURCES AND FULL METHOD

Every citation, the complete validation method, and the raw numbers behind sections 3 and 5.
lumos-evidence.vercel.app

Project Lumos

Every schema change ships an upgrader · the AI never emits coordinates · one logical edit is one undo step